

Precision Reuse for Exchange between Verifiers



Márk Somorjai, Marie-Christine Jakobs
LMU Munich

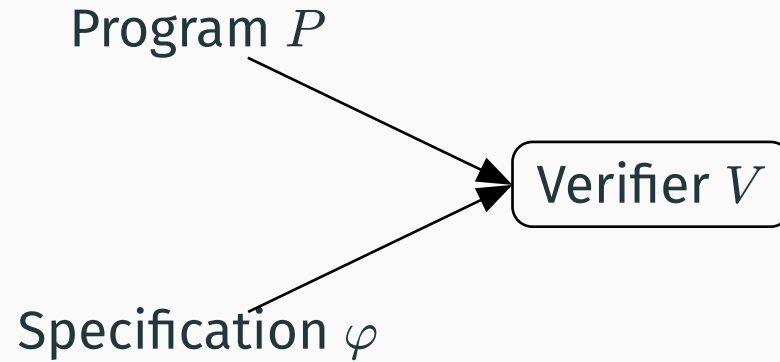
AVM 2026
2026-09-23

Check whether a **program** P fulfills some **specification** φ .

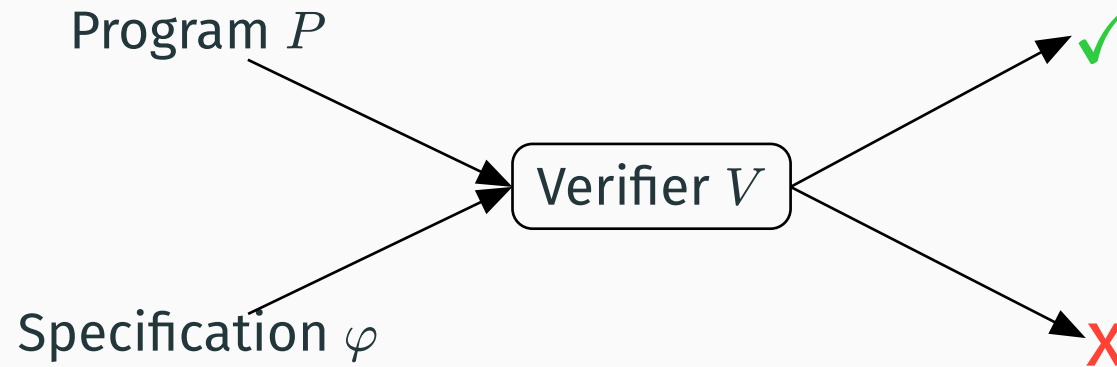
Program P

Specification φ

Check whether a **program** P fulfills some **specification** φ .

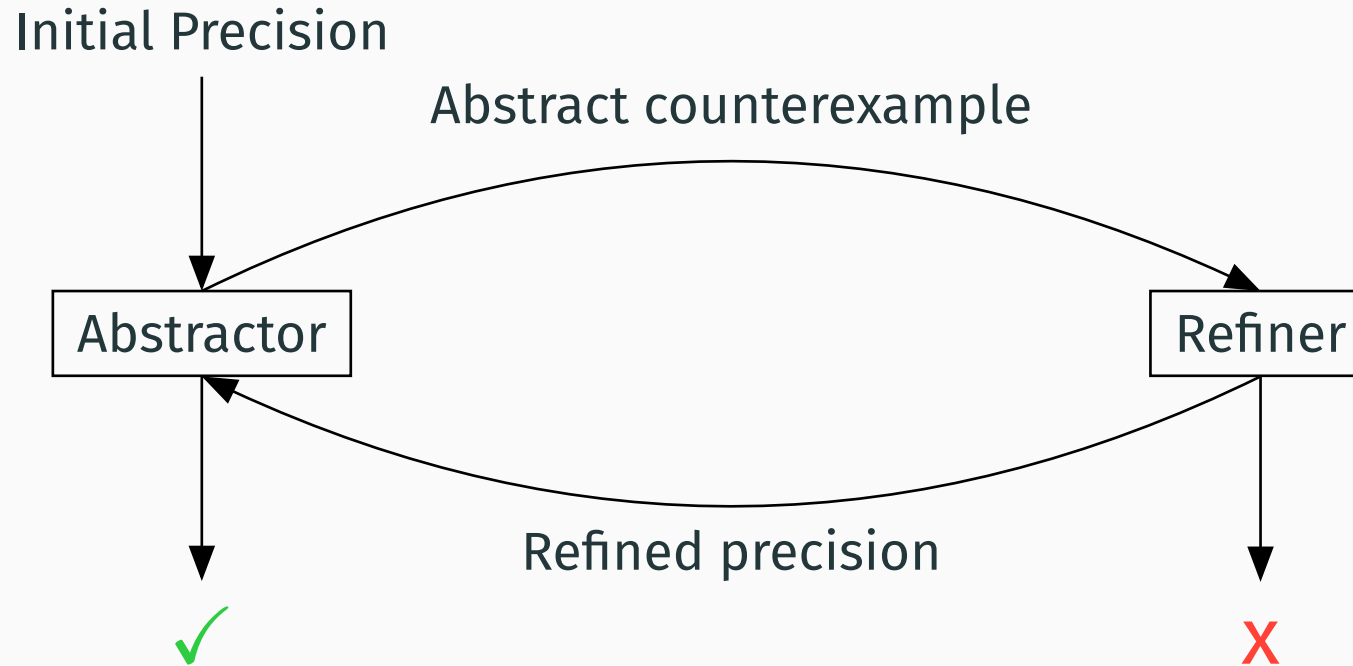


Check whether a **program** P fulfills some **specification** φ .



Specification can be:

- reachability
- memory safety
- termination



Iteratively refine level of abstraction (**precision**) until:

- ✓ the precision is sufficient to prove specification
- ✗ a feasible counterexample is found

- Finding the right precision can be costly
 - Incremental software evolution
- **verifying each version is expensive**

- Finding the right precision can be costly
 - Incremental software evolution
- **verifying each version is expensive**

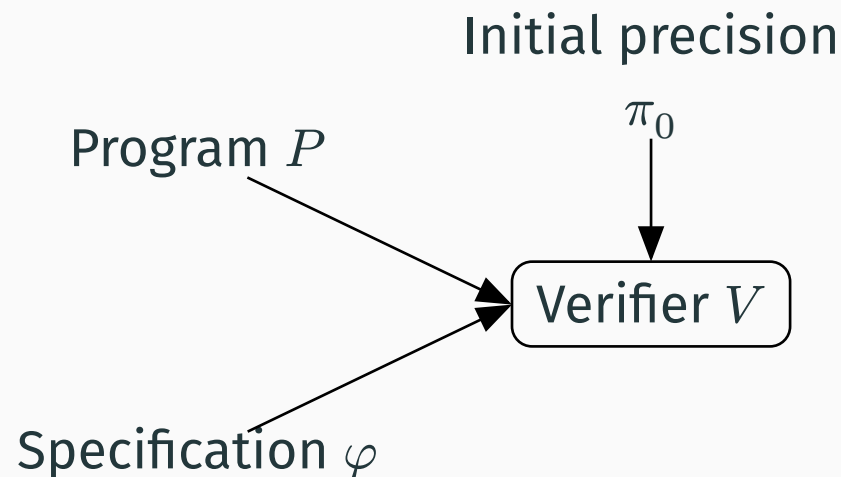
Precision Reuse @ FSE 2013 [1]

Reuse **precision of the previous program** version's verification

- Finding the right precision can be costly
 - Incremental software evolution
- **verifying each version is expensive**

Precision Reuse @ FSE 2013 [1]

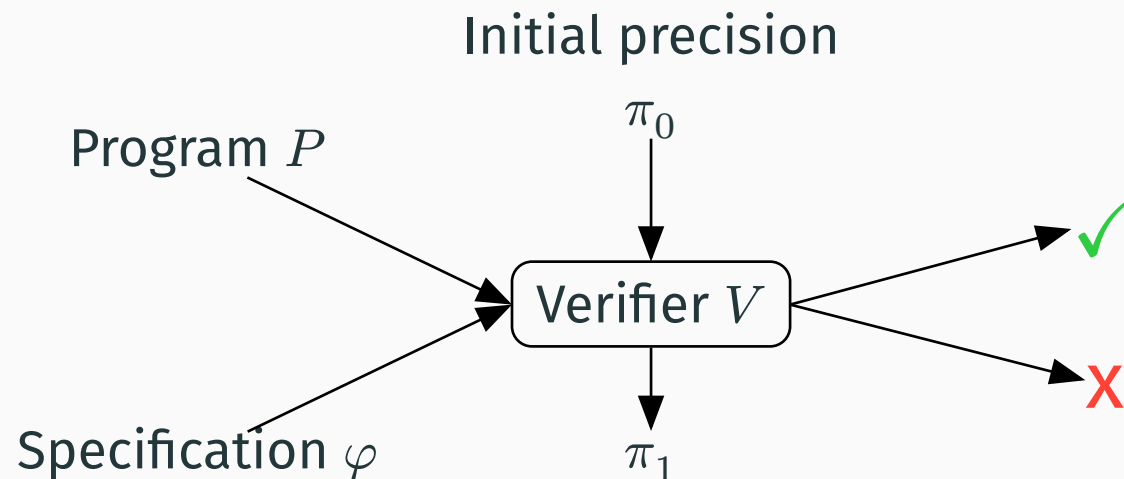
Reuse **precision of the previous program** version's verification



- Finding the right precision can be costly
 - Incremental software evolution
- **verifying each version is expensive**

Precision Reuse @ FSE 2013 [1]

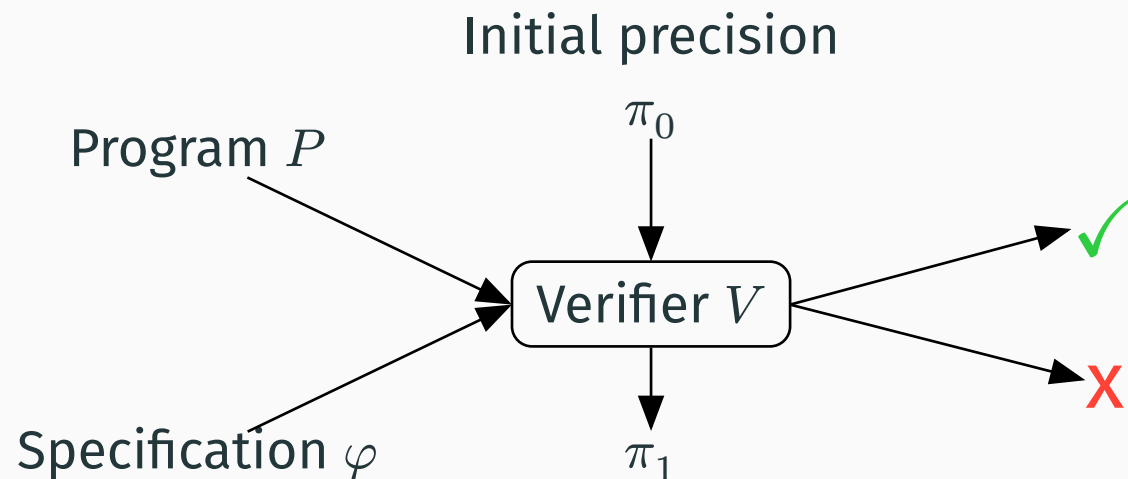
Reuse **precision of the previous program** version's verification



- Finding the right precision can be costly
 - Incremental software evolution
- **verifying each version is expensive**

Precision Reuse @ FSE 2013 [1]

Reuse **precision of the previous program** version's verification

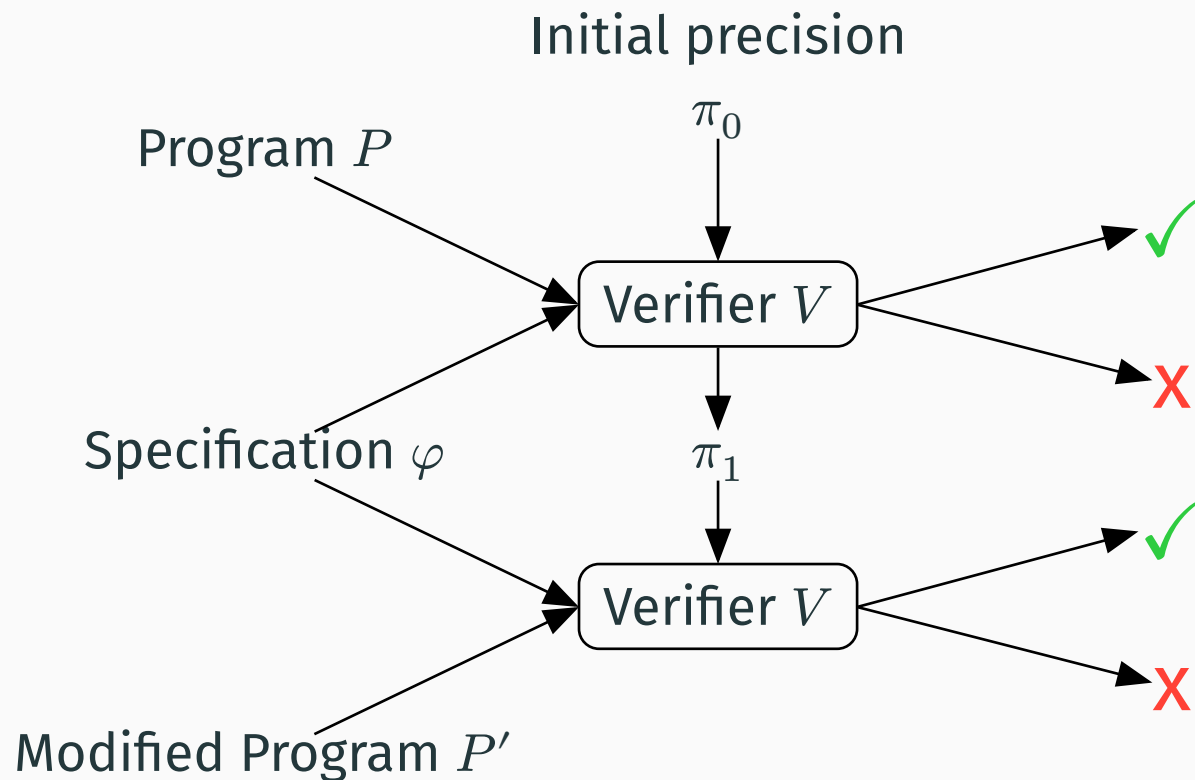


Modified Program P'

- Finding the right precision can be costly
 - Incremental software evolution
- **verifying each version is expensive**

Precision Reuse @ FSE 2013 [1]

Reuse **precision of the previous program** version's verification



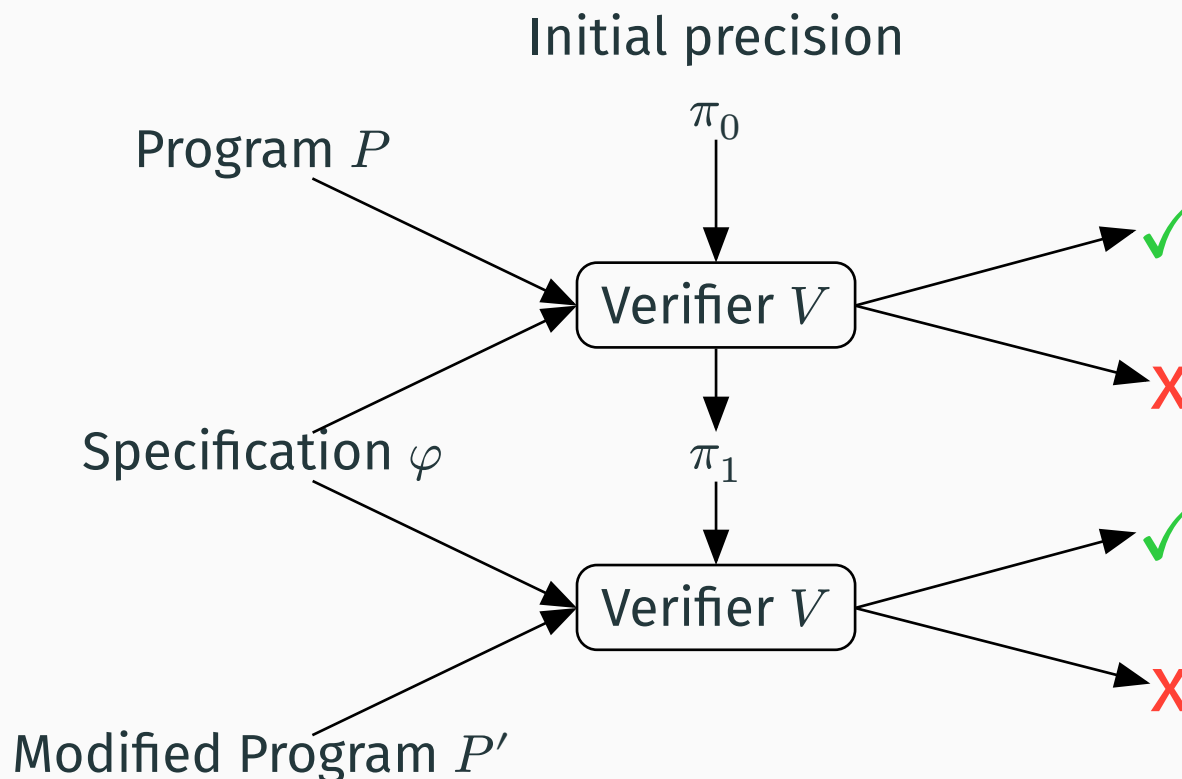
- Finding the right precision can be costly
 - Incremental software evolution
- **verifying each version is expensive**

Precision Reuse @ FSE 2013 [1]

Reuse **precision of the previous program** version's verification

- can help **reduce #refinements**
- maintains **soundness**

→ **improve efficiency**



D. Beyer, S. Löwe, E. Novikov, A. Stahlbauer, P. Wendler:
Precision reuse for efficient regression verification [1]

Realizability claims: easy to integrate into abstraction-based verifiers

Performance claims: ~4x analysis time speedup

D. Beyer, S. Löwe, E. Novikov, A. Stahlbauer, P. Wendler:
Precision reuse for efficient regression verification [1]

Realizability claims: easy to integrate into abstraction-based verifiers

Performance claims: ~4x analysis time speedup

No other tool has adopted it since, though:

- evolving codebases are still a **challenge for applicability** of verification [2]
- **many** abstraction-based verification tools exist (23 of 45 @ SV-COMP '26 [3])

D. Beyer, S. Löwe, E. Novikov, A. Stahlbauer, P. Wendler:
Precision reuse for efficient regression verification [1]

Realizability claims: easy to integrate into abstraction-based verifiers

Performance claims: ~4x analysis time speedup

No other tool has adopted it since, though:

- evolving codebases are still a **challenge for applicability** of verification [2]
- **many** abstraction-based verification tools exist (23 of 45 @ SV-COMP '26 [3])

Reproduction and replication study with Marie-Christine Jakobs

- realizability: **easy to implement**, but format **not tool-independent**
- performance: still **improved**, but **less gains** (~1.4x analysis time speedup)
 - 18-95% less refinements by tools compared to 2013

Precision reuse: share precisions between **verification runs**

	same program version	different program versions
same tool	reverification ✓	regression verification ✓

Precision reuse: share precisions between **verification runs**

	same program version	different program versions
same tool	reverification ✓	regression verification ✓
different tools	cooperative verification	cooperative regression verification

Precision reuse: share precisions between **verification runs**

	same program version	different program versions
same tool	reverification ✓	regression verification ✓
different tools	cooperative verification	cooperative regression verification

Tool-specific format

Realizability: ? not tool-independent

- **proprietary format** using SMT-LIB
- **internal variables** of tools
- scoping **not well-defined**

Precision reuse: share precisions between **verification runs**

	same program version	different program versions
same tool	reverification ✓	regression verification ✓
different tools	cooperative verification	cooperative regression verification

Tool-specific format

Realizability: ? not tool-independent

- **proprietary format** using SMT-LIB
- **internal variables** of tools
- scoping **not well-defined**

Exchange format

Marian: let's use the witness format!

- uses the **program's language**
- uses **program variables**
- scoping with **program locations**

1 entry type: `precision_set`

- metadata
- content: list of precision elements

1 entry type: `precision_set`

- metadata
- content: list of precision elements

2 types of elements:

- predicates
- memory locations

1 entry type: `precision_set`

- metadata
- content: list of precision elements

2 types of elements:

- predicates
- memory locations

3 scoping options:

- global
- function
- local

1 entry type: precision_set

- metadata
- content: list of precision elements

2 types of elements:

- predicates
- memory locations

3 scoping options:

- global
- function
- local

Tool-specific format

```
1 (declare-fun |main::s| () Int)
2
3 main:
4 (assert (<= (mod |main::s|
5             4294967296) 0))
```

Exchange format

yaml

```
1 precision:
2   format: "c_expression"
3   scope:
4     type: "function"
5     function_name: "main"
6   type: "predicate"
7   values:
8     - "s % 4294967296 >= 0"
```

Tool-specific format

```
1 N17:  
2 x, ptr
```

Exchange format

yaml

```
1 precision:  
2   format: "c_expression"  
3   scope:  
4     type: "location"  
5     location:  
6       line: 14  
7       column: 3  
8   type: "relevant_memory_locations"  
9   values:  
10  - "&x"  
11  - "ptr"
```

Tools:

- **CPACHECKER** from LMU Munich, Germany [4]
- **SLOWBEAST** from Masaryk University, Czech Republic [5]
- **THETA** from Budapest University of Technology and Economics, Hungary [6]

Tools:

- **CPACHECKER** from LMU Munich, Germany [4]
- **SLOWBEAST** from Masaryk University, Czech Republic [5]
- **THETA** from Budapest University of Technology and Economics, Hungary [6]

Dataset: all C programs from SV-COMP

Tools:

- **CPACHECKER** from LMU Munich, Germany [4]
- **SLOWBEAST** from Masaryk University, Czech Republic [5]
- **THETA** from Budapest University of Technology and Economics, Hungary [6]

Dataset: all C programs from SV-COMP

Claims

- 1 **Format Comparison:** Exchange format **as good as** tool-specific format
- 2 **Improved Performance:** More effective and efficient, less refinements
- 3 **Cooperation:** Sharing precisions between tools **verifies new programs**

Reverification using precisions in **exchange vs. tool-specific format**

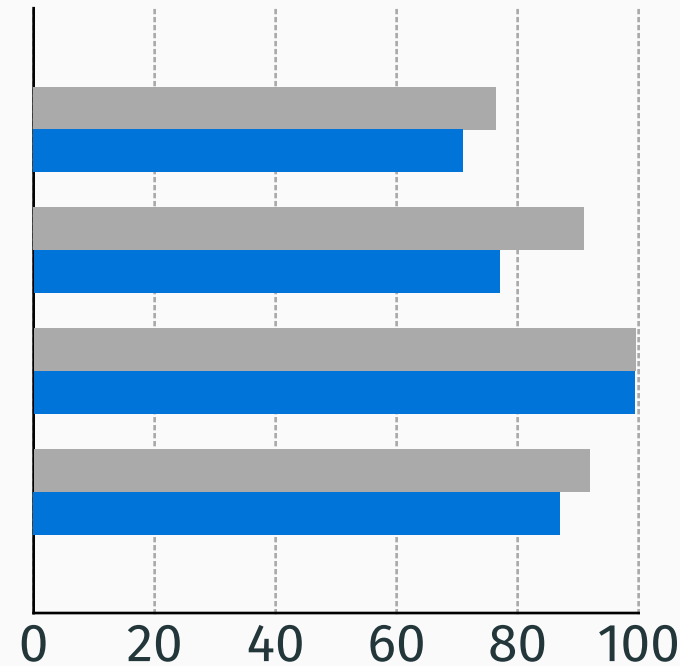
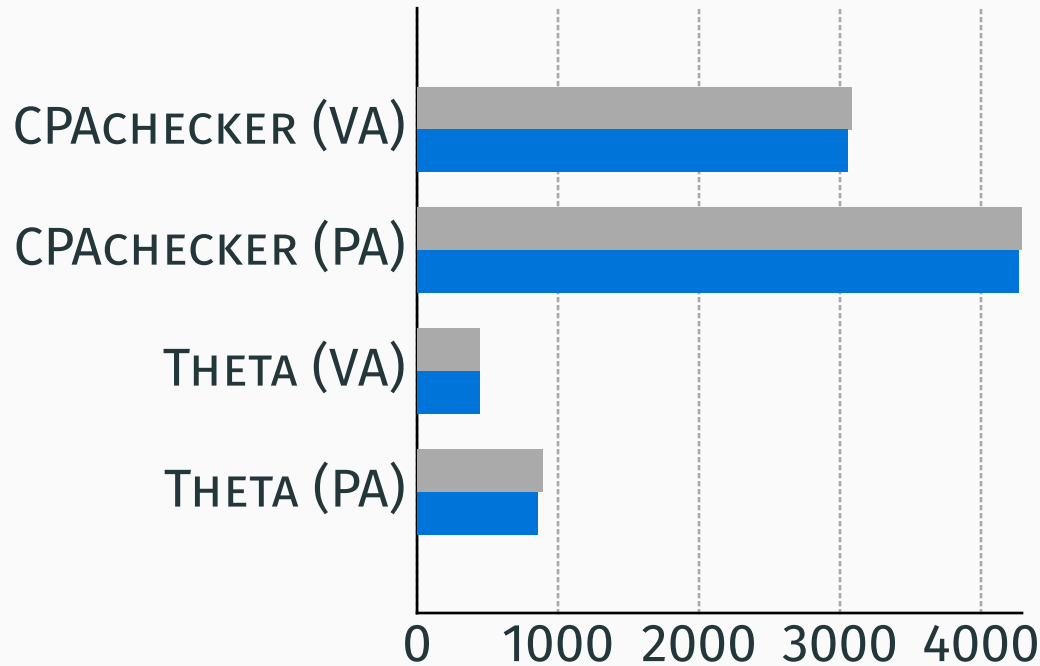
Claim 1 – Format Comparison

Reverification using precisions in **exchange vs. tool-specific format**

Verified programs

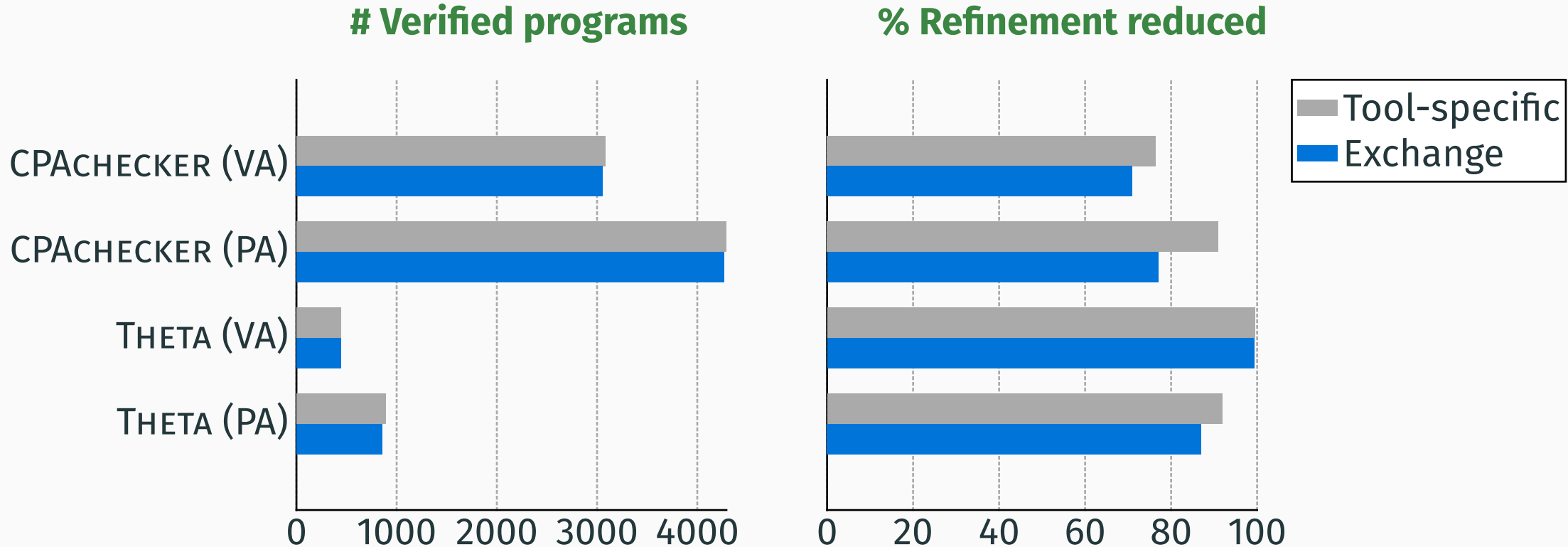
% Refinement reduced

Tool-specific
Exchange



Claim 1 – Format Comparison

Reverification using precisions in **exchange vs. tool-specific format**



There is **no significant difference** between the formats

Cooperative verification:



Cooperative verification:



Only consider programs where:

- **Producer** required >1 refinement
- **Consumer** required >1 refinement

Cooperative verification:



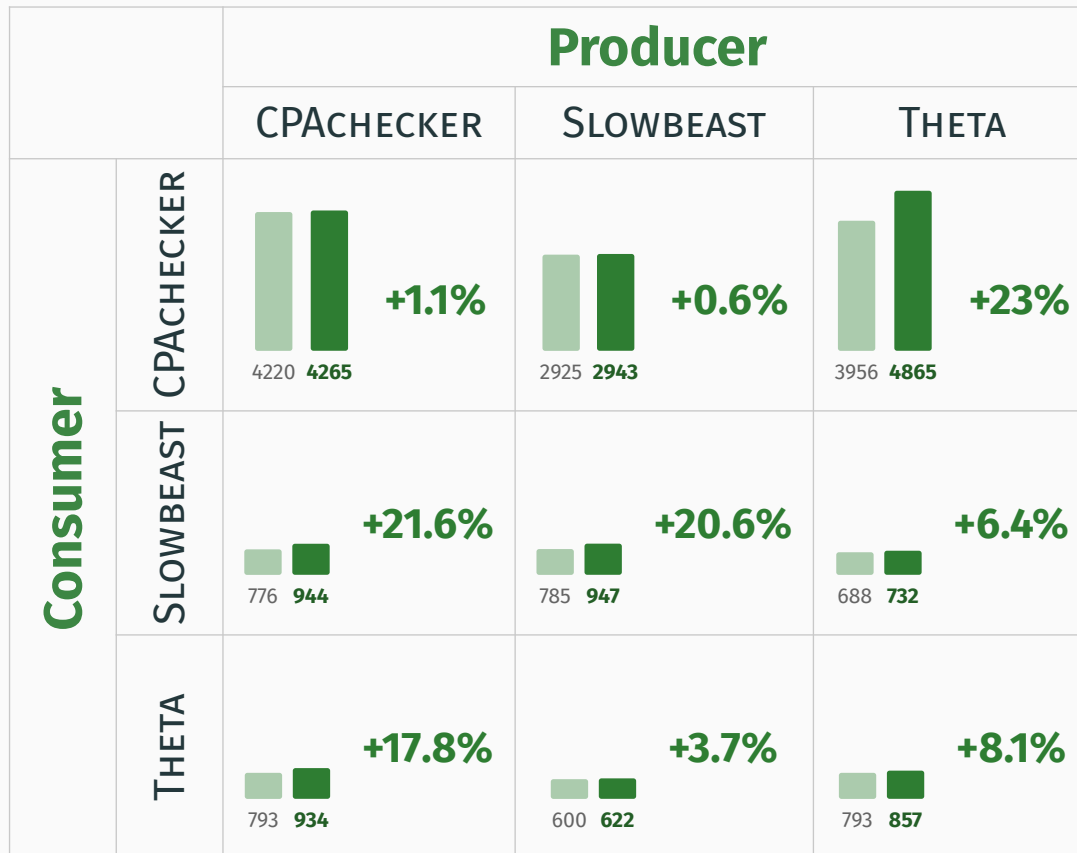
Only consider programs where:

- **Producer** required >1 refinement
- **Consumer** required >1 refinement

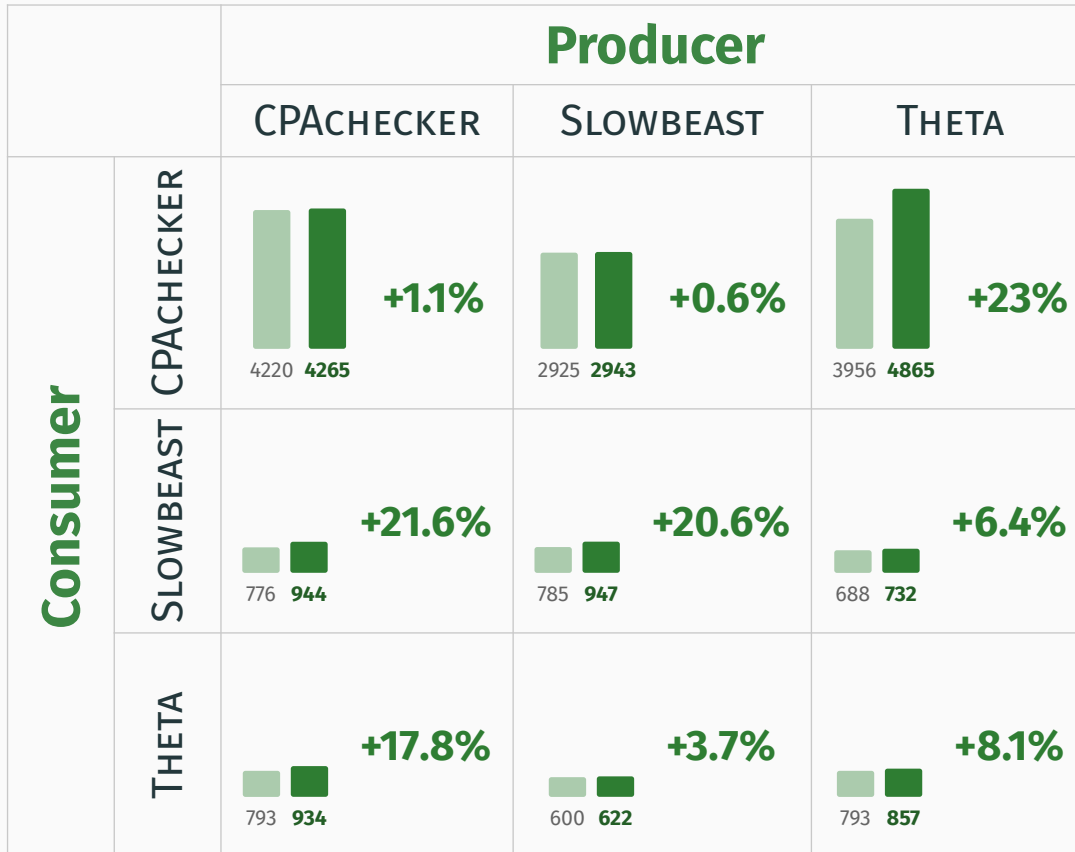
Evaluation criteria:

- **Effectiveness**: #programs verified
- **Efficiency**: verification time
- **Refinements**: #refinements

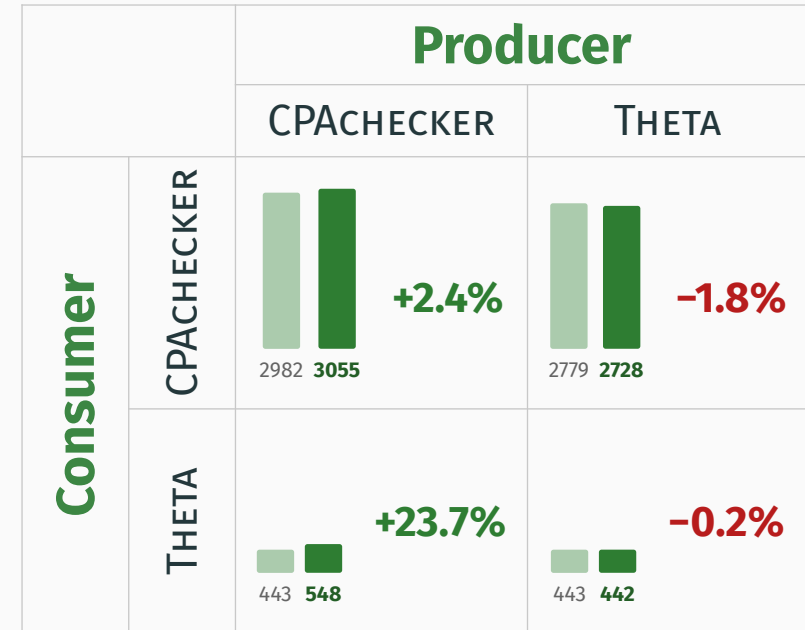
Predicate Analysis



Predicate Analysis



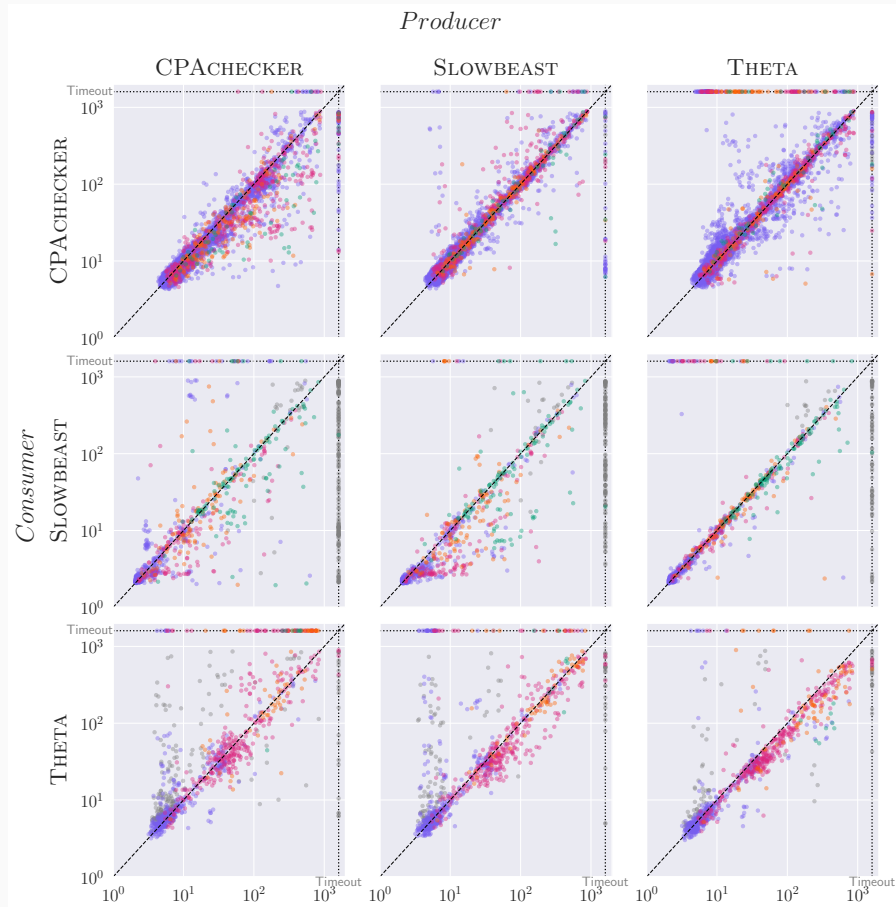
Value Analysis



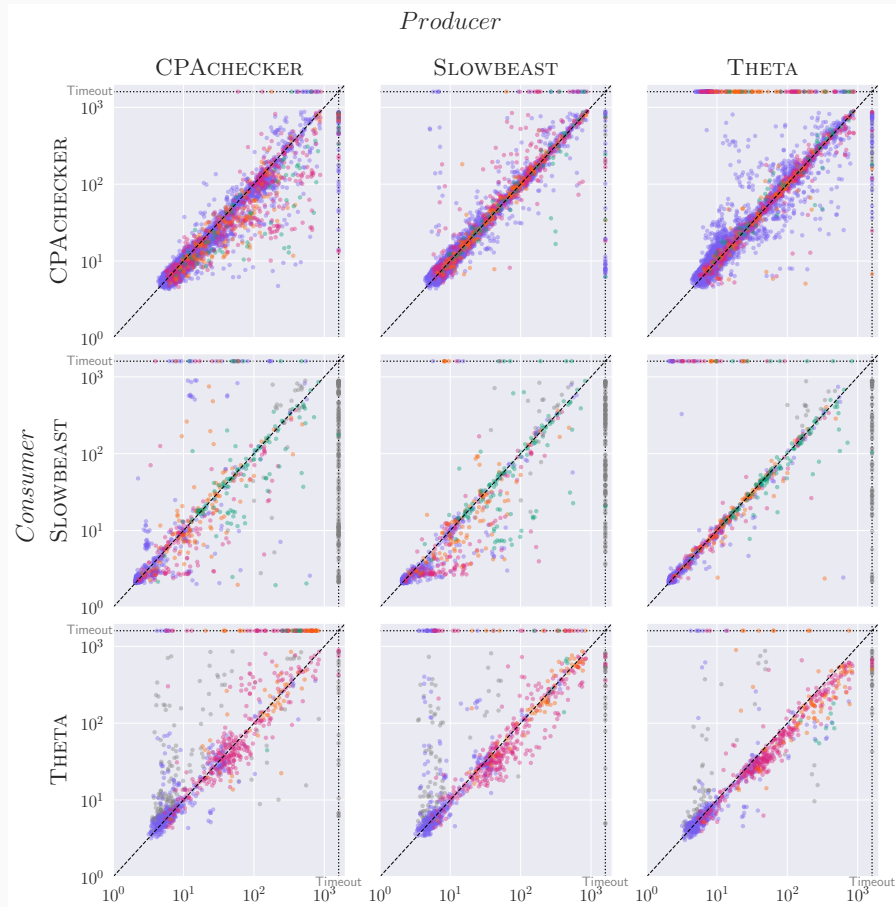
without reuse

with reuse

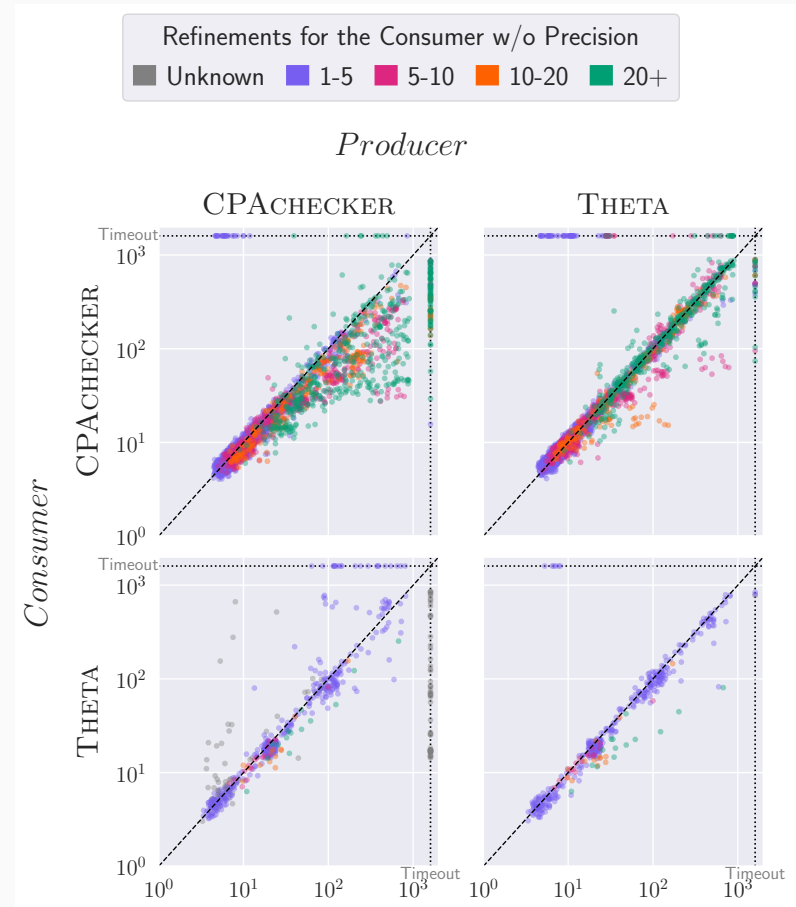
Predicate Analysis



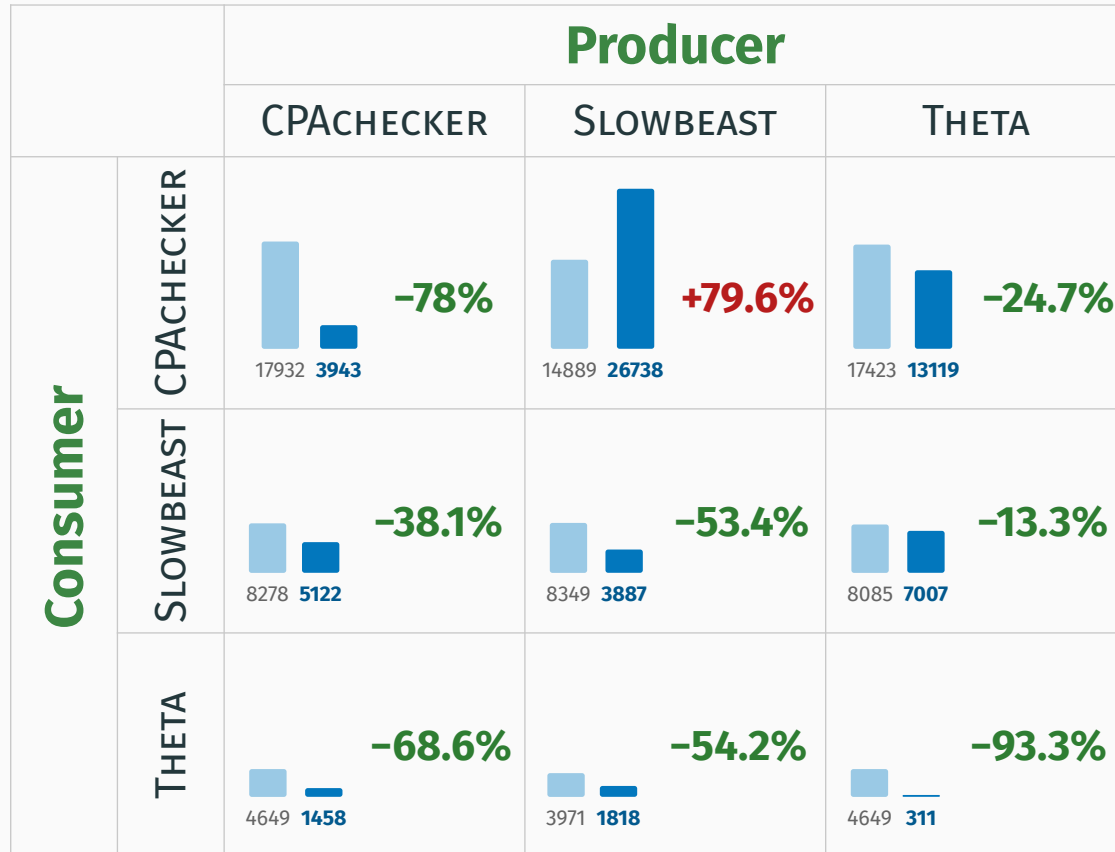
Predicate Analysis



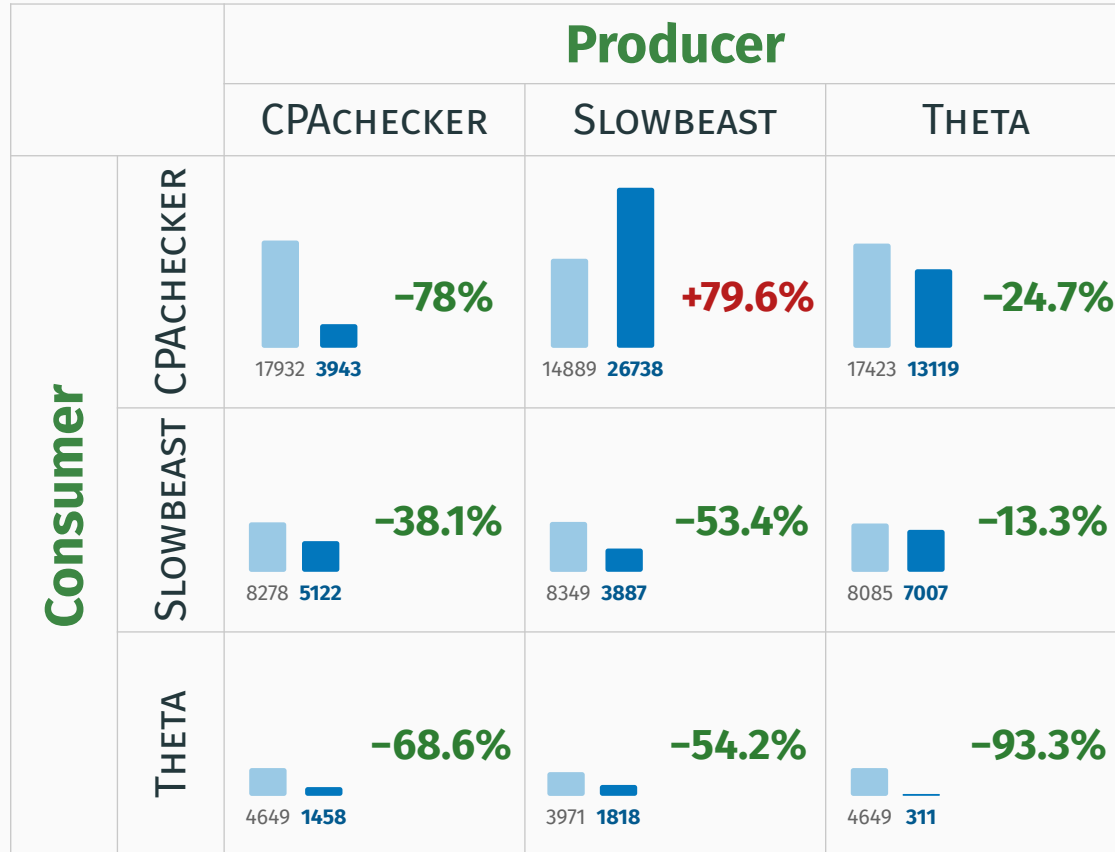
Value Analysis



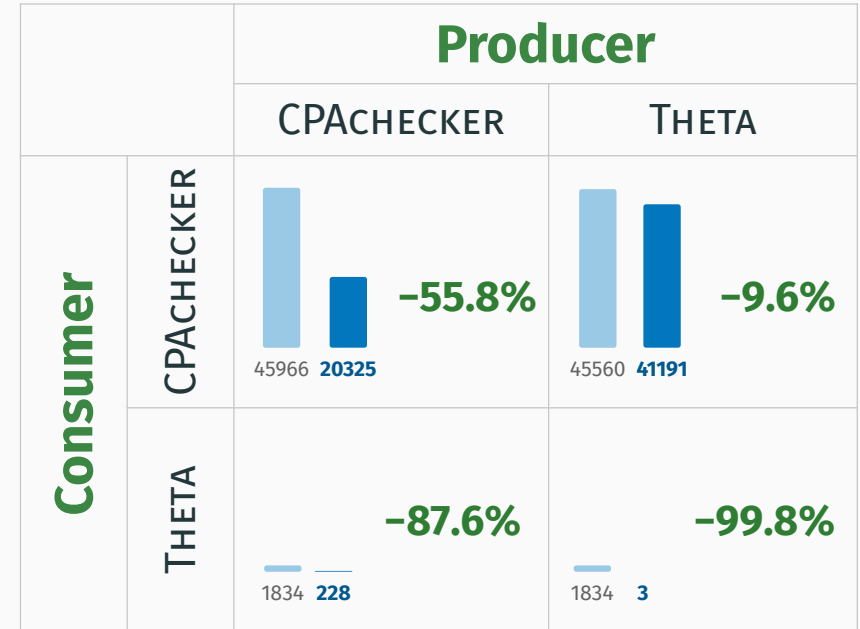
Predicate Analysis



Predicate Analysis



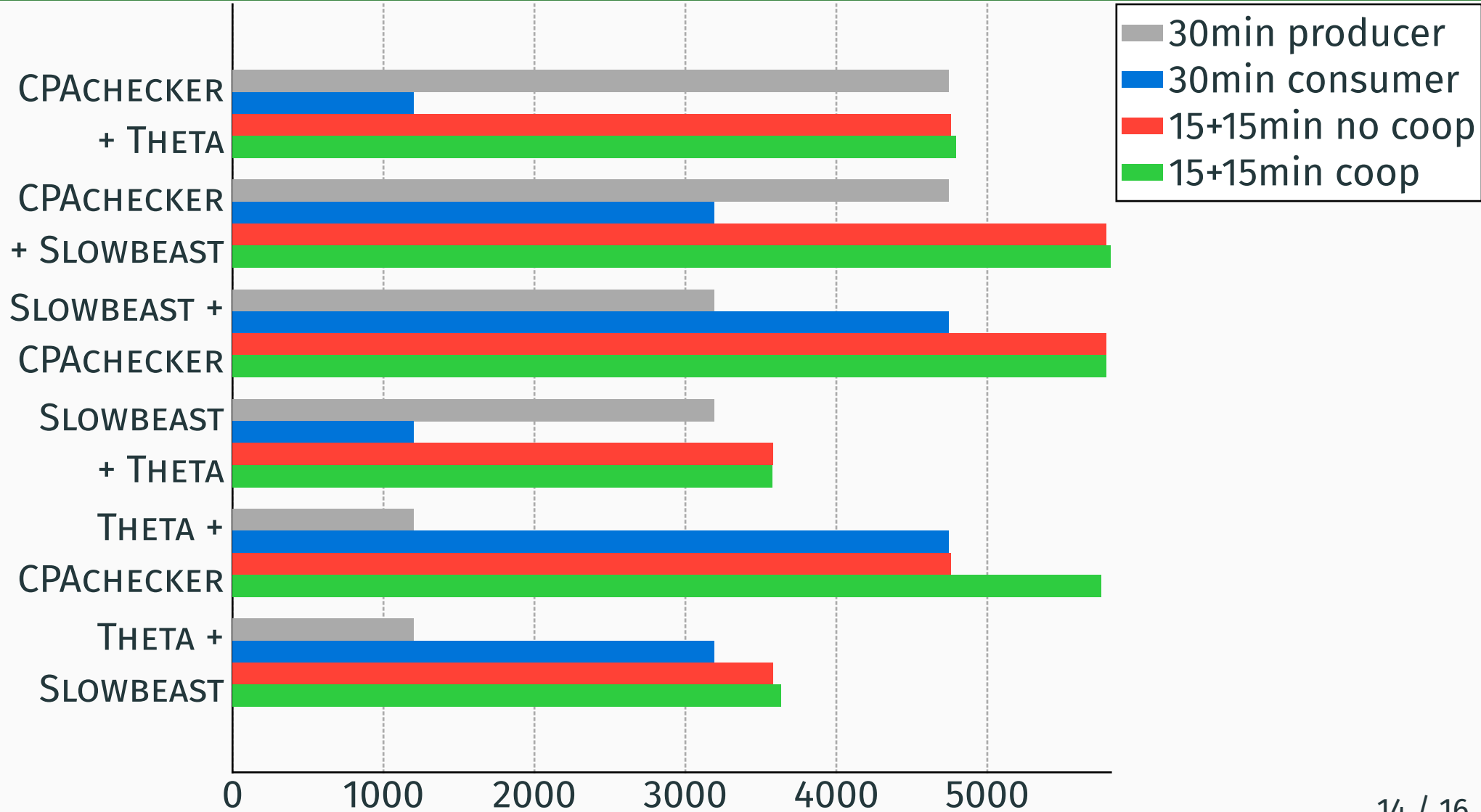
Value Analysis



without reuse

with reuse

Claim 3 – Cooperation



Claim 3 – Cooperation

Unique

(69)

(35)

(10)

(50)

(1159)

(63)

CPACHECKER

+ THETA

CPACHECKER

+ SLOWBEAST

SLOWBEAST +

CPACHECKER

SLOWBEAST

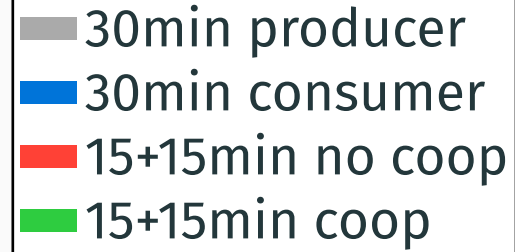
+ THETA

THETA +

CPACHECKER

THETA +

SLOWBEAST



0 1000 2000 3000 4000 5000

Reproduction and replication study of **precision reuse**

- **Easy to integrate** into abstraction-based verifiers
- **Positive effects** are not as strong, but still **observed**

Reproduction and replication study of **precision reuse**

- **Easy to integrate** into abstraction-based verifiers
- **Positive effects** are not as strong, but still **observed**

Precision Exchange Format as witness format extension

- Similar performance to the tool-specific format
- Exchange of precisions **improves performance**
- **Cooperation** through precisions can **verify new programs**

Reproduction and replication study of **precision reuse**

- **Easy to integrate** into abstraction-based verifiers
- **Positive effects** are not as strong, but still **observed**

Precision Exchange Format as witness format extension

- Similar performance to the tool-specific format
- Exchange of precisions **improves performance**
- **Cooperation** through precisions can **verify new programs**

Future work

- Further tool support?
- Cooperative regression verification

Reproduction and replication study of **precision reuse**

- **Easy to integrate** into abstraction-based verifiers
- **Positive effects** are not as strong, but still **observed**

Precision Exchange Format as witness format extension

- Similar performance to the tool-specific format
- Exchange of precisions **improves performance**
- **Cooperation** through precisions can **verify new programs**

Future work

- Further tool support?
- Cooperative regression verification

Study



Format



- [1] D. Beyer, S. Löwe, E. Novikov, A. Stahlbauer, and P. Wendler, “Precision reuse for efficient regression verification,” in *Proc. FSE*, ACM, 2013, pp. 389–399. [Online]. Available: <https://doi.org/10.1145/2491411.2491429>
- [2] P. W. O'Hearn, “Continuous Reasoning: Scaling the impact of formal methods,” in *Proc. LICS*, ACM, 2018, pp. 13–25. [Online]. Available: <https://doi.org/10.1145/3209108.3209109>
- [3] D. Beyer and J. Strejcek, “Evaluating Software Verifiers for C, Java, and SV-LIB,” in *Tools and Algorithms for the Construction and Analysis of Systems*, S. Junges and G. Katz, Eds., Springer Nature Switzerland, 2026, pp. 461–502.
- [4] D. Beyer and M. E. Keremoglu, “CPAchecker: A Tool for Configurable Software Verification,” in *Proc. CAV*, in LNCS 6806. Springer, 2011, pp. 184–190. [Online]. Available: https://doi.org/10.1007/978-3-642-22110-1/_16
- [5] M. Jonáš, J. Strejček, and A. Griggio, “Combining Symbolic Execution with Predicate Abstraction and CEGAR,” in *Proc. FMCAD*, IEEE, 2024, pp. 271–280. doi: [10.34727/2024/ISBN.978-3-85448-065-5_33](https://doi.org/10.34727/2024/ISBN.978-3-85448-065-5_33).
- [6] Á. Hajdu and Z. Micskei, “Efficient Strategies for CEGAR-Based Model Checking,” *JAR*, vol. 64, no. 6, pp. 1051–1091, 2020, [Online]. Available: <https://doi.org/10.1007/s10817-019-09535-x>